

# upctl-compose 用户手册

本地部署 & AI Agent 自动处理指南

基于 AI Agent 的全自动开发平台

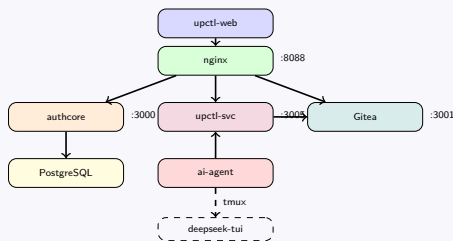
馨翼

2026 年 8 月 31 日

# 目录

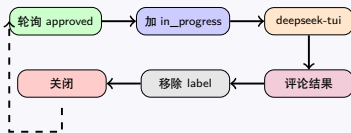
1. 系统架构
2. 本地部署
3. 使用指南
4. 组件日志
5. 容器内 CI
6. 故障排除

# 系统架构总览



| 服务         | 端口        | 作用                              |
|------------|-----------|---------------------------------|
| nginx      | 8088      | 反向代理，路由到各后端                     |
| authcore   | 3000      | 身份认证，JWT 签发                     |
| upctl-svc  | 3005      | 工单 CRUD API                     |
| upctl-web  | (nginx 内) | Vue 3 前端                        |
| ai-agent   | —         | 轮询 Gitea + tmux 托管 deepseek-tui |
| Gitea      | 3001      | 代码托管 + 工单 + CI                  |
| PostgreSQL | 5432      | 数据库                             |

## AI Agent 工作流程



### 关键特征

每 5 分钟轮询 | tmux 托管 deepseek-tui | 自动评论并关闭

# 环境要求

## 软件要求

- Docker 24.0+, Compose 2.24+
- Git

## 端口占用

|                 |                |
|-----------------|----------------|
| 5432 PostgreSQL | 3000 authcore  |
| 3001 Gitea      | 3005 upctl-svc |
| 8088 nginx      | 2222 Gitea SSH |

## 推荐配置

- CPU 4 核 +、内存 8 GB +
- 需访问 GitHub + DeepSeek API

**注意：** Docker Desktop 需配置 no\_proxy（已内置）

# 克隆与启动

## 1. 克隆

```
git clone https://github.com/  
alchemy-studio/upctl-compose.git  
cd upctl-compose
```

## 2. 启动

```
docker compose up -d  
docker compose ps
```

## 启动日志

```
authcore: Running migration...  
[authcore] Starting htyuc...  
数据库连接池初始化 OK  
upctl-svc: starting on port 3005  
ai-agent: Setting up workspace...  
Starting tmux session...
```

# 初始化 Gitea

## 等待 Gitea

```
until curl -sf localhost:3001/api/v1/version  
do sleep 3; done
```

## 初始化

```
docker compose exec ai-agent \  
python3 /app/setup-gitea.py
```

## 创建内容

组织 weli + 仓库 tickets

标签: approved(1) / in\_progress(2) / blocked(3)

## 管理界面

<http://localhost:3001>

用户 ai-bot / ai-bot-dev-pass

# AI 代理配置

## .env 文件

```
DEEPSEEK_API_KEY=sk-your-key  
DEEPSEEK_API_BASE=https://api.deepseek.com  
DEFAULT_MODEL=deepseek-v4-flash
```

## 重启

```
docker compose up -d ai-agent
```

## 验证部署

```
curl -sf localhost:8088/ | head -c 200  
curl -sf localhost:3005/ # upctl-svc  
curl -sf localhost:3000/ # AuthCore
```



## 登录页面

访问 `http://localhost:8088`

- 开发模式显示用户名密码登录表单
- 使用 demo / demo123 登录
- AuthCore 验证后直接签发 JWT
- 登录后跳转工单列表

**认证流程：** 前端 → nginx → authcore 验证密码  
→ JWT



## 工单列表 & 详情

列表页：标签页切换 / 卡片显示标题 + 状态  
详情页：标题 + 编号 / 状态标签 / 评论区

AI 处理结果格式：

## Processing result

{AI 总结}

--- \*Processed by DeepSeek Agent\*

The screenshot displays a web application for managing tickets. At the top, there's a header with the title '工单列表' (Ticket List) and a '筛选' (Filter) button. Below the header, there are three tabs: '待处理' (Pending), '已关闭' (Closed), and '全部' (All). The main content area shows a list of tickets. Two tickets are visible:

- Ticket #24: 数据库连接池优化 (Database connection pool optimization), status '待处理' (Pending), created at 05-09 11:35.
- Ticket #23: 登录页面样式调整 (Login page style adjustment), status '待处理' (Pending), created at 05-09 11:35.

Below the ticket list, there's a code editor showing the implementation of the 'login\_required' decorator in a Django view named 'app/orders/views.py'.

```
#!/usr/bin/env python
# coding: utf-8

"""
python
import csv
from django.http import HttpResponse
from django.contrib.auth.decorators import login_required
from models import WorkOrder

@login_required
def export_workorders_csv(request):
    status = request.GET.get('status', '')
    orders = WorkOrder.objects.all()
    if status:
        orders = orders.filter(status=status)

    response = HttpResponse(content_type='text/csv; charset=utf-8')
    response['Content-Disposition'] = 'attachment; filename="workorders.csv"'

    writer = csv.writer(response)
    writer.writerow(['ID', 'Title', 'Status', 'Assigned To', 'Created At', 'Updated At'])
    for order in orders:
        writer.writerow([order.id, order.title, order.get_status_display(),
            order.assigned_to.username if order.assigned_to else '',
            order.created_at, order.updated_at])
```

## Gitea 后台

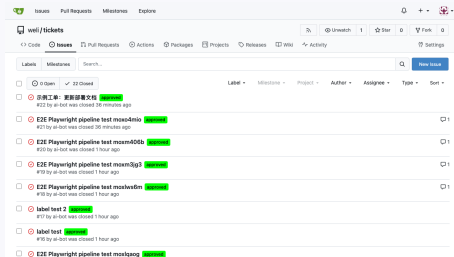
地址 `http://localhost:3001`

用户 `ai-bot` / `ai-bot-dev-pass`

仓库 `weli/tickets`

标签说明：

- `approved` — AI 自动处理
- `in_progress` — 处理中
- `blocked` — 阻塞



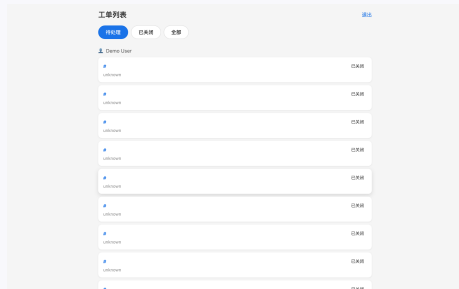
## AI Agent 自动处理

系统部署完成后，AI Agent 自动开始轮询：

- 1 在 Gitea 创建工单并打 approved 标签
- 2 ai-agent 每 5 分钟轮询发现新工单
- 3 自动调用 DeepSeek API 处理
- 4 评论结果并关闭工单

也可手动启动即时处理：

```
$ docker compose exec ai-agent \  
python3 /app/poll_worker.py --once
```



ai-agent 自动轮询日志：

```
Processing issue #185  
DeepSeek response received  
close #185: 200 -- completed
```

# 各组件日志解读

## authcore 启动

```
Running migration 2022-02-14-124946
Running migration 2026-05-02-082153
[authcore] Starting htyuc...
全局数据库连接池初始化 OK
```

## upctl-svc

```
[upctl-svc] starting on port 3005
[upctl-svc] listening on 0.0.0.0:3005
```

## ai-agent 轮询

```
poll_worker started - interval=300s
No approved issues found.
No work. Sleeping 300s...
```

## ai-agent 处理

```
Processing issue #185: 添加 dependbot
add_label #185: 200
DeepSeek response received (594 chars)
add_comment #185: 200
close #185: 200 - completed
```

## tmux 运行时 & 容器状态

### tmux 调试

```
# 进入 session
docker compose exec ai-agent
  tmux attach -t deepseek-agent
# 查看输出
docker compose exec ai-agent
  tmux capture-pane -t deepseek-agent -p
```

### docker compose ps

```
NAME STATUS
ai-agent Up (healthy)
authcore Up
gitea Up
postgres Up (healthy)
upctl-svc Up
upctl-web Up
```

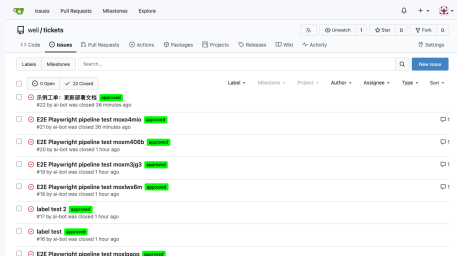
### 使用 tmux send-keys 注入命令

可在不进入交互模式的情况下向 agent 发送命令

# Gitea Issue 自动处理流程

本系统使用 Gitea + ai-agent 实现工单自动处理，无需外部 CI：

- 1 创建工单，标记 approved
- 2 ai-agent 每 5min 轮询 Gitea API
- 3 发现 approved 工单 → 加 in\_progress
- 4 tmux send-keys 注入工单内容到 deepseek-tui 对话
- 5 deepseek-tui 与 api.deepseek.com 交互生成回复
- 6 tmux capture-pane 读取回复内容
- 7 评论结果到工单
- 8 移除 in\_progress + 关闭工单



## ai-agent 工作日志

### 启动阶段

```
Setting up workspace...  
Starting tmux session...  
Starting poll worker...  
poll_worker started - interval=300s
```

### 空闲状态

```
No approved issues found.  
No work. Sleeping 300s...
```

### 处理工单

```
Processing issue #185: 添加 dependbot  
add_label #185: 200  
DeepSeek response received (594 chars)  
add_comment #185: 200  
close #185: 200 - completed
```

**关键技术：** Python 轮询 | tmux send-keys 注入 | deepseek-tui CLI | Gitea API



# deepseek-tui 验证

## tmux 终端

```
root@ai-agent:/app/workspace# deepseek-tui --version
deepseek-tui (npm wrapper) v0.8.22
binary version: v0.8.22
repo: git+https://github.com/Hmbown/DeepSeek-TUI.git
```

**注意：** 日志已脱敏，无密文泄露

## 实际运行日志

```
docker compose logs ai-agent --tail 5
[ai-agent] Setting up workspace...
[ai-agent] Starting tmux session...
[ai-agent] Starting poll worker...
[2026-05-09] poll_worker started -- interval=300s
[2026-05-09] No work. Sleeping 300s...
```

## 常见问题

| 问题                    | 原因                | 解决                             |
|-----------------------|-------------------|--------------------------------|
| POOL_SIZE must be set | authcore 缺变量      | 加 POOL_SIZE: "10"              |
| JWT_KEY not set       | upctl-svc 缺变量     | 加 JWT_KEY 环境变量                 |
| Gitea API 404         | 路径错误              | 用 /api/v1/ 根路径                 |
| 前端白屏/JS 404           | 资源未构建             | docker compose build upctl-web |
| ai-agent 未连 DeepSeek  | 缺 API Key         | 配置后重启                          |
| 容器间连接失败               | Docker Desktop 代理 | 设置 no_proxy (已内置)              |
| Playwright 401        | JWT 格式            | ts 无 Z 后缀                      |
| remove_label 404      | 标签 ID             | approved=1, in_progress=2      |

# 总结

## upctl-compose AI Agent 驱动的全自动开发平台

### 核心能力



6 容器一键部署



DeepSeek 自动处理



Playwright 全链路测试

```
git clone https://github.com/alchemy-studio/upctl-compose.git && cd  
upctl-compose && docker compose up -d
```

# Thank You

F馨F翼

<https://github.com/alchemy-studio/upctl-compose>